01

MGU BCA - 3RD SEM

Design and Analysis of Algorithms

Module 1 - Introduction and Analysis

By Aben Sojan

Exam - 12 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

EXAM PATTERN (from real MGU question papers)

DAA Honours exam: 50 marks, 1 hour 30 minutes. Part A: 10 x 2 marks, all compulsory, short questions from all modules. Part B: 3 x 5 marks, focused explanations with examples. Part C: 1 x 15 marks, full algorithm trace or essay. Total: 20 + 15 + 15 = 50. Part C clusters on MST (Prim's/Kruskal's), knapsack, Floyd-Warshall, N-Queens.

TOPIC 1

What is an Algorithm?

1

An algorithm is a step-by-step procedure to solve a problem in a finite number of steps.

Every algorithm must have: input, output, definiteness (each step clear), finiteness (it must stop), and effectiveness (steps are doable).

Example: a cooking recipe is an algorithm; Euclid's GCD method is an algorithm.

Design techniques include divide and conquer, greedy, dynamic programming and backtracking (Modules 2 and 3).

QUESTION BANK

1 Define an algorithm. What are its characteristics?

SURE MARK (May 2019, Oct 2025 H)

An algorithm is a finite sequence of well-defined, unambiguous steps that solves a problem or performs a computation.

It takes zero or more inputs and produces at least one output, with a clear relationship between them.

Characteristic 1, Input: zero or more quantities supplied to the algorithm from outside.

Characteristic 2, Output: at least one quantity produced as the result.

Characteristic 3, Definiteness: every step must be precisely defined, leaving no room for guessing.

Characteristic 4, Finiteness: the algorithm must terminate after a finite number of steps for every input.

Characteristic 5, Effectiveness: every step must be basic enough to be carried out in principle, even with pencil and paper.

Example: adding two numbers - read a, read b, compute c = a + b, print c.

Each step is clear, it always stops, and it always gives the right sum.

In short: an algorithm is a correct, clear and terminating recipe for solving a problem.

2 What are the different algorithm design techniques? Name them.

REPEATED (Oct 2021, Feb 2025)

Algorithm design techniques are general strategies for building algorithms; this syllabus uses four of them.

Divide and conquer: split the problem into smaller subproblems of the same type, solve them recursively, and combine the answers.

Example: merge sort.

Greedy: build the answer step by step, always taking the locally best choice and never reconsidering it.

Example: Dijkstra's shortest path.

Dynamic programming: solve overlapping subproblems once, store the answers and reuse them instead of recomputing.

Example: Floyd Warshall all-pairs shortest path.

Backtracking: build candidates step by step and abandon a partial candidate as soon as it cannot lead to a valid answer.

Example: N-Queens.

Choosing the right technique is the first step of algorithm design; Modules 2 and 3 study each one in depth.

3 What is the difference between an algorithm and a program?

IMPORTANT

An algorithm is the idea; a program is that idea written down in a programming language so a machine can run it.

An algorithm is described in plain language or pseudocode and is machine independent, while a program is written in Python, C or Java for a real computer.

The same algorithm can become many different programs in different languages.

An algorithm must always terminate (finiteness is compulsory), but a program need not - an operating system is meant to run forever.

An algorithm is analysed for correctness and time/space complexity; a program is additionally tested, debugged and maintained.

Example: binary search written as pseudocode is the algorithm; the same logic typed into Python is the program.

In short: every program implements some algorithm, but an algorithm exists even before any code is written.

TOPIC 2

Space Complexity

2

Space complexity is the total memory an algorithm needs: fixed part plus variable part.

Fixed part: code size and constants.

Variable part: depends on input size n, like arrays and recursion stack.

Example: summing n numbers needs O(n) for the array plus O(1) extra variables.

Recursive algorithms use extra stack space, one frame per recursive call.

QUESTION BANK

4 **Differentiate space complexity and time complexity of algorithms with an example.**

SURE MARK (May 2019, Oct 2021, Feb 2025)

Time complexity is the number of basic steps an algorithm takes, expressed as a function of the input size n.

Space complexity is the amount of memory an algorithm needs, also expressed as a function of n.

Time is counted in operations such as comparisons and assignments; space is counted in variables, arrays and call-stack frames.

Both are written with asymptotic notation so that we compare growth rates, not exact counts.

Example: iterative binary search takes O(log n) time and O(1) extra space, but the recursive version needs O(log n) stack space.

Example: merge sort takes O(n log n) time and O(n) extra space for the temporary merge arrays.

There is often a trade-off: using more memory, like a lookup table, can buy a faster running time.

In short: time is how long it runs, space is how much memory it needs - analyse both before choosing an algorithm.

5 Define space complexity. What are its components?

IMPORTANT

Space complexity is the amount of memory an algorithm needs, expressed as a function of the input size n.

It has two components.

Fixed space: memory for the program code itself and for constants; it does not change with input size.

Variable space: memory that grows with the input - space for the input data, for extra variables and data structures, and for the recursion call stack.

Example: adding n numbers stored in an array needs O(n) space for the array plus O(1) for the loop variables.

Recursive algorithms add one stack frame per active call, so recursion depth directly adds to variable space.

We usually count only the extra workspace beyond the input, and we express it with Big-O, e.g. O(1) or O(n).

A good algorithm balances time and space: merge sort is fast but needs O(n) extra space, while quick sort sorts in place.

6 Why does recursion increase space complexity?

IMPORTANT

Every active function call needs memory for its local variables, parameters and return address; this memory lives in a stack frame on the call stack.

A recursive call cannot finish until the calls it made finish, so all frames from the first call down to the deepest one stay on the stack at the same time.

Therefore a recursion of depth d needs O(d) extra stack space on top of everything else the algorithm uses.

Example: recursive factorial(5) keeps 5 frames alive at the deepest point, so it uses O(n) stack space for input n.

The iterative version of the same loop uses only O(1) extra space, since no calls pile up.

This is also why very deep recursion crashes with a stack overflow: the stack memory runs out.

In short: recursion trades elegance for stack memory, and depth directly becomes space.

TOPIC 3

Time Complexity

3

Time complexity counts the number of basic operations an algorithm performs as input size n grows.

We care about growth rate, not exact seconds, so it is machine independent.

Cases: best case (minimum), average case, worst case (maximum, most used for guarantees).

Example: linear search worst case checks all n items, so O(n).

QUESTION BANK

7 **Define time complexity. Explain best, average and worst cases.**

SURE MARK (May 2019, Oct 2021, Feb 2025)

Time complexity measures how the running time of an algorithm grows when the input size n grows.

We count basic operations (comparisons, assignments) instead of seconds, so the measure is machine independent.

Best case: the minimum time over all inputs of size n.

Example: linear search finding the key at the first position takes O(1).

Average case: the expected time over typical inputs; it needs assumptions about how inputs are distributed.

Worst case: the maximum time over all inputs of size n.

Example: linear search scanning all n items takes O(n).

The worst case is used most often because it gives a guarantee - the algorithm will never be slower than this.

For linear search on n items: best O(1), average O(n/2) which is O(n), worst O(n).

Reporting all three cases gives the complete picture, but the worst case is the standard promise of an algorithm.

8 What is performance analysis of an algorithm? Explain.

REPEATED (May 2019, Oct 2021)

Performance analysis of an algorithm means measuring how its running time and memory use grow as the input size n grows.

It has two parts: time complexity (the number of steps) and space complexity (the memory used).

Analysis is done theoretically by counting basic operations, not by timing on a machine, so the result is machine independent.

Growth is expressed with asymptotic notation such as Big-O; for example linear search is O(n) time and O(1) extra space.

Example: for sorting n numbers, bubble sort is O(n^2) while merge sort is O(n log n) - analysis tells us merge sort wins for large n without running either.

We usually analyse the worst case, because it guarantees how the algorithm behaves on its hardest input.

In short: performance analysis lets us compare algorithms on paper before writing any code.

9 Why do we analyse worst-case time complexity?

IMPORTANT

The worst case gives a hard guarantee: no input of size n can make the algorithm slower than this bound.

Users and system designers plan with guarantees - a video player must decode every frame in time, not just the average frame.

The average case needs probability assumptions about inputs, and real inputs are often not random; a wrong assumption makes the analysis useless.

The best case is usually trivial and tells us nothing about how the algorithm behaves on hard inputs.

For many algorithms the worst case is also easy to find and analyse, e.g. quick sort on an already sorted array.

That is why textbooks and interviews quote the worst case by default: it is safe, assumption-free and informative.

TOPIC 4

4

Asymptotic Notations: Big-O, Omega, Theta

Asymptotic notations describe growth rates ignoring constants and small inputs.

Big-O, O(g(n)): upper bound. f(n) = O(g(n)) if f(n) <= c*g(n) for all n >= n0, for some c > 0.

Omega, Omega(g(n)): lower bound. f(n) >= c*g(n) for all n >= n0.

Theta, Theta(g(n)): tight bound. c1*g(n) <= f(n) <= c2*g(n); both upper and lower bound hold.

Common order: O(1) < O(log n) < O(n) < O(n log n) < O(n^2) < O(2^n).

We drop constants and lower-order terms: 3n^2 + 5n + 2 is O(n^2).

QUESTION BANK

10 **Define Big-O notation with its mathematical definition.**

REPEATED (Feb 2025, Oct 2025 H)

Big-O notation gives an asymptotic UPPER bound: it says a function never grows faster than some simpler function g(n), ignoring constant factors.

Mathematical definition: f(n) = O(g(n)) if there exist positive constants c and n0 such that f(n) <= c * g(n) for all n >= n0.

The constant c absorbs multiplicative factors, and n0 says the bound only needs to hold for large inputs.

Worked example: show 3n + 2 = O(n).

Need 3n + 2 <= c*n for n >= n0.

For n >= 2, 3n + 2 <= 3n + n = 4n, so c = 4 and n0 = 2 work.

Intuition: for large n, 3n + 2 behaves like n up to a constant factor, so it is O(n).

Big-O lets us compare algorithms by growth rate alone: O(n log n) always beats O(n^2) for large n.

11 Define Omega and Theta notations.

REPEATED (Feb 2025, Oct 2025 H)

Omega notation gives an asymptotic LOWER bound: the function grows at least as fast as g(n).

Definition: f(n) = Omega(g(n)) if there exist c > 0 and n0 such that f(n) >= c * g(n) for all n >= n0.

Theta notation gives a TIGHT bound: the function grows at the same rate as g(n), trapped between two constant multiples.

Definition: f(n) = Theta(g(n)) if there exist c1 > 0, c2 > 0 and n0 such that c1*g(n) <= f(n) <= c2*g(n) for all n >= n0.

In other words, Theta means both the O and the Omega bound hold with the same g(n).

Example: f(n) = 5n^2 + 3n is Theta(n^2), because for n >= 1 we have 5n^2 <= f(n) <= 8n^2.

Memory aid: Big-O is at most, Omega is at least, Theta is exactly this fast in growth terms.

12 Show that 2n^2 + 10n is O(n^2).

IMPORTANT

We must find constants c > 0 and n0 such that 2n^2 + 10n <= c*n^2 for all n >= n0.

For n >= 1 we have n <= n^2, so 10n <= 10n^2.

Substituting: 2n^2 + 10n <= 2n^2 + 10n^2 = 12n^2 for all n >= 1.

So choosing c = 12 and n0 = 1 satisfies the Big-O definition.

Hence 2n^2 + 10n = O(n^2), as required.

The constants are not unique: c = 3 and n0 = 10 also work, since 10n <= n^2 whenever n >= 10.

The lower-order term 10n is swallowed by n^2 for large n, which is exactly what Big-O captures.

13 Arrange in increasing order of growth: O(n^2), O(1), O(n log n), O(2^n), O(log n), O(n).

IMPORTANT

Growth rates ordered from slowest to fastest give the standard complexity ladder.

O(1): constant - the time does not change no matter how big n gets.

O(log n): logarithmic - grows very slowly, e.g. binary search halving the input.

O(n): linear - time grows in direct proportion to n, e.g. a single scan.

O(n log n): linearithmic - slightly worse than linear, e.g. merge sort.

O(n^2): quadratic - doubling n quadruples the time, e.g. bubble sort.

O(2^n): exponential - adding 1 to n doubles the time, e.g. naive Fibonacci recursion.

Final increasing order: O(1) < O(log n) < O(n) < O(n log n) < O(n^2) < O(2^n).

Memory trick: constant, log, linear, linearithmic, quadratic, exponential.

This ladder is the scale every complexity comparison in the subject is measured against.

14 What is the difference between Big-O and Theta?

IMPORTANT

Big-O is only an upper bound: it says the algorithm grows no faster than g(n), but it may grow much slower.

Theta is a tight bound: it pins the growth from both above and below, describing the exact growth rate.

Example: binary search is O(n) - a true but loose statement - and Theta(log n), which is tight and informative.

Saying an algorithm is O(n^2) leaves open whether it is really quadratic or actually linear; Theta(n) would settle the question.

In practice we quote Big-O for worst-case guarantees and Theta when we have proved the exact behaviour.

Prefer Theta whenever you can prove it; Big-O is the safe fallback that is always available.

TOPIC 5

Recursion: Basic Concept

5

Recursion is when a function calls itself to solve smaller versions of the same problem.

Every recursive solution needs: a base case (stops the calls) and a recursive case (smaller problem).

Example: factorial(n) = n * factorial(n-1), with factorial(0) = 1 as base case.

Recursion is natural for divide and conquer, tree and graph traversal, and backtracking.

QUESTION BANK

15 What is recursion? Explain with an example.

IMPORTANT

Recursion is a technique where a function calls itself on a smaller version of the same problem, until it reaches a trivial base case.

Every recursive definition needs two parts: a base case whose answer is known directly, and a recursive case that shrinks the problem toward the base case.

Pseudocode for factorial:

```
factorial(n):
if n == 0: return 1 # base case
```

```
return n * factorial(n-1) # recursive case
```

Trace of factorial(3): 3 * factorial(2), which is 3 * 2 * factorial(1), which is 3 * 2 * 1 * factorial(0) = 3 * 2 * 1 * 1 = 6.

Each call waits for the smaller call to finish, then combines the result; the call stack remembers the pending multiplications.

Recursion mirrors the structure of trees, divide-and-conquer and backtracking problems, which is why it appears everywhere in this subject.

16 What happens if a recursive function has no base case?

IMPORTANT

Without a base case the function keeps calling itself on smaller inputs forever; this is infinite recursion.

Every call pushes a new stack frame, so the call stack grows without limit until the program crashes with a stack overflow.

Example: factorial without the n == 0 check would call factorial(-1), factorial(-2), and so on until memory runs out.

Even with a base case present, the recursive step must actually move toward it; calling factorial(n+1) would also never terminate.

So two rules: always include a base case, and always shrink the input on each recursive call.

The base case is the emergency brake of recursion - no brake, no stopping, and the program crashes.

TOPIC 6

Analysis of Recursive Algorithms

6

Recursive running time is written as a recurrence relation, e.g. T(n) = 2T(n/2) + n.

Substitution method: guess the answer, then prove it by induction.

Recursion tree method: draw the tree of calls and add the cost at each level.

Master method (shortcut for divide and conquer): for T(n) = aT(n/b) + f(n), compare f(n) with n^(log_b a).

QUESTION BANK

17 What is a recurrence relation? Give an example.

IMPORTANT

A recurrence relation expresses the running time T(n) for input size n in terms of the running time for smaller inputs, plus the cost of the current step.

It always comes with one or more base cases that give T directly for tiny inputs.

Example from merge sort: T(n) = 2T(n/2) + n, with T(1) = 1.

Reading it: sorting n items costs the time to sort two halves (2T(n/2)) plus n for merging them.

Solving the recurrence - by substitution, recursion tree or the Master method - gives the closed form, here T(n) = Theta(n log n).

Recurrences are the standard language for analysing recursive algorithms, and every divide-and-conquer algorithm yields one.

18 Explain the substitution method for solving recurrences.

IMPORTANT

The substitution method solves a recurrence in two steps: guess the form of the answer, then prove the guess correct by mathematical induction.

Step 1, guess: use experience or a recursion tree to propose a bound, e.g. T(n) = O(n log n) for T(n) = 2T(n/2) + n.

Step 2, prove: assume the bound holds for all smaller sizes, i.e. T(k) <= c*k*log(k) for k < n, and show it follows for n.

```
T(n) = 2T(n/2) + n <= 2*(c*(n/2)*log(n/2)) + n = c*n*log(n) - c*n + n <= c*n*log(n) for c >= 1.
```

The induction closes, so the guess is proved and T(n) = O(n log n) is established.

The method needs a good initial guess; drawing a recursion tree first usually supplies it.

Substitution is the rigorous proof technique behind most recurrence solutions expected in exams.

19 Solve T(n) = 2T(n/2) + n using a recursion tree.

IMPORTANT

A recursion tree draws every recursive call as a node labelled with its non-recursive cost, arranged level by level.

Level 0: one node for the whole problem of size n, costing n.

```
Level 1: two nodes of size n/2, each costing n/2, for a level total of n.
```

```
Level 2: four nodes of size n/4, each costing n/4, for a level total of n.
```

In general, level i has 2^i nodes of size n/2^i, each costing n/2^i, so every level costs exactly n.

The tree stops when the subproblem size reaches 1, which happens after log2(n) levels, since n/2^i = 1 gives i = log n.

Total cost = (cost per level) x (number of levels) = n x log n.

Hence T(n) = Theta(n log n); this is the classic merge-sort analysis every student must be able to reproduce.

The tree makes the n log n visible: log n levels, each doing n work.